Using forms

When you don't want to rerun your script with each input made by a user, `st.form` is here to help! Forms make it easy to batch user input into a single rerun. This guide to using forms provides examples and explains how users interact with forms.

Example

In the following example, a user can set multiple parameters to update the map. As the user changes the parameters, the script will not rerun and the map will not update. When the user submits the form with the button labeled "**Update map**", the script reruns and the map updates.

If at any time the user clicks "**Generate new points**" which is outside of the form, the script will rerun. If the user has any unsubmitted changes within the form, these will *not* be sent with the rerun. All changes made to a form will only be sent to the Python backend when the form itself is submitted.

View source code

Built with Streamlit 🎈 Fullscreen

User interaction

If a widget is not in a form, that widget will trigger a script rerun whenever a user changes its value. For widgets with keyed input (`st.number_input`, `st.text_input`, `st.text_area`), a new value triggers a rerun when the user clicks or tabs out of the widget. A user can also submit a change by pressing `Enter` while their cursor is active in the widget.

On the other hand, if a widget is in a form, the script will not rerun when a user tabs out of that widget. If the form has a submit button, the script will rerun when the form is submitted and all widgets within the form will send their updated values to the Python backend.

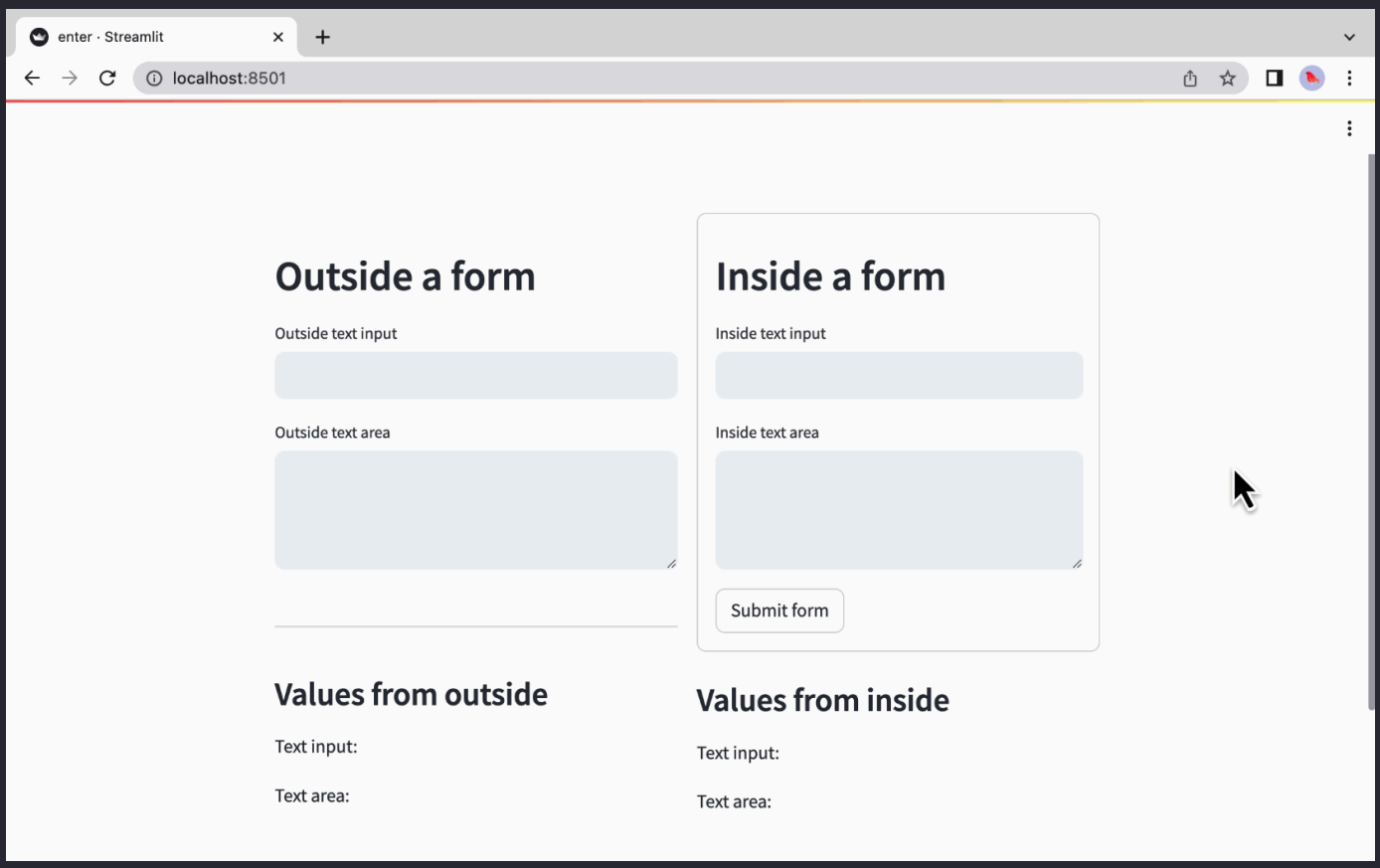
We use cookies to help us understand how you interact with our website. By clicking “Accept All Cookies” or closing this banner, you consent to our use of cookies. For more information, please see our Privacy Notice.

Reject All

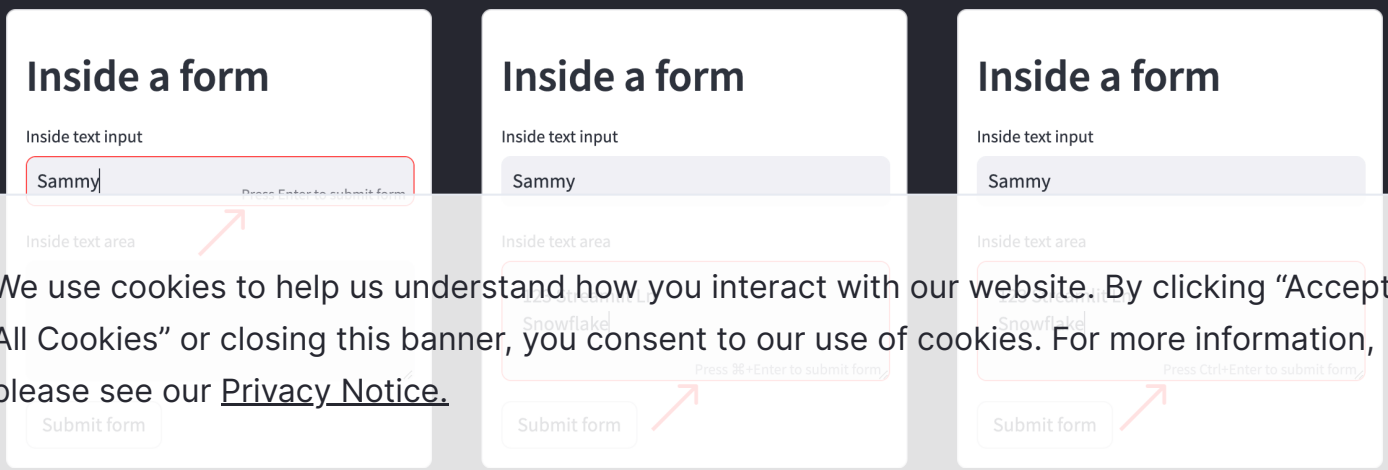
Accept All Cookies

Cookie settings

Ask AI



A user can submit a form using **Enter** on their keyboard if their cursor active in a widget that accepts keyed input. Within `st.number_input` and `st.text_input` a user presses **Enter** to submit the form. Within `st.text_area` a user presses **Ctrl+Enter/⌘+Enter** to submit the form.



We use cookies to help us understand how you interact with our website. By clicking “Accept All Cookies” or closing this banner, you consent to our use of cookies. For more information, please see our Privacy Notice.

Widget values

Before a form is submitted, all widgets within that form will have default values, just like widgets outside of a form have default values.

PYTHON

```python
import streamlit as st

with st.form("my_form"):
    st.write("Inside the form")
    my_number = st.slider('Pick a number', 1, 10)
    my_color = st.selectbox('Pick a color', ['red','orange','green','blue','
    st.form_submit_button('Submit my picks')

# This is outside the form
st.write(my_number)
st.write(my_color)
```

Built with Streamlit 🎈 Fullscreen

Forms are containers

When `st.form` is called, a container is created on the frontend. You can write to that container like you do with other container elements. That is, you can use Python's `with` statement as shown in the example above, or you can assign the form container to a variable and call methods on it directly. Additionally, you can place `st.form_submit_button` anywhere in the form container.

PYTHON

```python
import streamlit as st
animal = st.form('my_animal')

# This is writing directly to the main body. Since the form container is
# defined above, this will appear below everything written in the form.
sound = st.selectbox('Sounds like', ['meow','woof','squeak','tweet'])

# These methods called on the form container, so they appear inside the for
submit = animal.form_submit_button(f'Say it with {sound}!')
sentence = animal.text_input('Your sentence:', 'Where\'s the tuna?')
```

We use cookies to help us understand how you interact with our website. By clicking “Accept All Cookies” or closing this banner, you consent to our use of cookies. For more information, please see our Privacy Notice.

```
say_it = sentence.rstrip('.,!?') + f', {sound}!'
if submit:
    animal.subheader(say_it)
else:
    animal.subheader(' ')
```

Built with Streamlit 🎈 Fullscreen

Processing form submissions

The purpose of a form is to override the default behavior of Streamlit which reruns a script as soon as the user makes a change. For widgets outside of a form, the logical flow is:

1. The user changes a widget's value on the frontend.
2. The widget's value in `st.session_state` and in the Python backend (server) is updated.
3. The script rerun begins.
4. If the widget has a callback, it is executed as a prefix to the page rerun.
5. When the updated widget's function is executed during the rerun, it outputs the new value.

For widgets inside a form, any changes made by a user (step 1) do not get passed to the Python backend (step 2) until the form is submitted. Furthermore, the only widget inside a form that can have a callback function is the `st.form_submit_button`. If you need to execute a process using newly submitted values, you have three major patterns for doing so.

Execute the process after the form

If you need to execute a one-time process as a result of a form submission, you can condition that process on the `st.form_submit_button` and execute it after the form. If you need results from your process to display above the form, you can use containers to control where the form displays relative to your output.

We use cookies to help us understand how you interact with our website. By clicking “Accept All Cookies” or closing this banner, you consent to our use of cookies. For more information, please see our Privacy Notice.

PYTHON

```
import streamlit as st

col1,col2 = st.columns([1,2])
col1.title('Sum:')
```

```python
with st.form('addition'):
    a = st.number_input('a')
    b = st.number_input('b')
    submit = st.form_submit_button('add')

if submit:
    col2.title(f'{a+b:.2f}')
```

Built with Streamlit 🎈 Fullscreen

Use a callback with session state

You can use a callback to execute a process as a prefix to the script rerunning.

Important

When processing newly updated values within a callback, do not pass those values to the callback directly through the `args` or `kwargs` parameters. You need to assign a key to any widget whose value you use within the callback. If you look up the value of that widget from `st.session_state` within the body of the callback, you will be able to access the newly submitted value. See the example below.

PYTHON

```python
import streamlit as st

if 'sum' not in st.session_state:
    st.session_state.sum = ''
def sum():
    result = st.session_state.a + st.session_state.b
    st.session_state.sum = result

col1,col2 = st.columns(2)
col1.title('Sum:')
if isinstance(st.session_state.sum, float):
```

We use cookies to help us understand how you interact with our website. By clicking “Accept All Cookies” or closing this banner, you consent to our use of cookies. For more information, please see our Privacy Notice.

```
    col2.title(f'{st.session_state.sum:.2f}')

with st.form('addition'):
    st.number_input('a', key = 'a')
    st.number_input('b', key = 'b')
    st.form_submit_button('add', on_click=sum)
```

Built with Streamlit 🎈 Fullscreen

Use `st.rerun`

If your process affects content above your form, another alternative is using an extra rerun. This can be less resource-efficient though, and may be less desirable that the above options.

PYTHON

```
import streamlit as st

if 'sum' not in st.session_state:
    st.session_state.sum = ''

col1,col2 = st.columns(2)
col1.title('Sum:')
if isinstance(st.session_state.sum, float):
    col2.title(f'{st.session_state.sum:.2f}')

with st.form('addition'):
    a = st.number_input('a')
    b = st.number_input('b')
    submit = st.form_submit_button('add')

# The value of st.session_state.sum is updated at the end of the script rer
# so the displayed value at the top in col2 does not show the new sum. Trig
# a second rerun when the form is submitted to update the value above.
st.session_state.sum = a + b
if submit:
    st.rerun()
```

We use cookies to help us understand how you interact with our website. By clicking “Accept All Cookies” or closing this banner, you consent to our use of cookies. For more information, please see our Privacy Notice.

Built with Streamlit 🎈 Fullscreen

Limitations

- Every form must contain a `st.form_submit_button`.
- `st.button` and `st.download_button` cannot be added to a form.
- `st.form` cannot be embedded inside another `st.form`.
- Callback functions can only be assigned to `st.form_submit_button` within a form; no other widgets in a form can have a callback.
- Interdependent widgets within a form are unlikely to be particularly useful. If you pass `widget1`'s value into `widget2` when they are both inside a form, then `widget2` will only update when the form is submitted.

← **Previous:** Session State

Next: Fragments →

Still have questions?

Our forums are full of helpful information and Streamlit experts.

We use cookies to help us understand how you interact with our website. By clicking “Accept All Cookies” or closing this banner, you consent to our use of cookies. For more information, please see our Privacy Notice.